المدرسة الوطنية للعلوم التطبيقية
ⵜⵉⵎⵙⵙⵓⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵙⴰⵏⵉⵡⵉⵏ ⵜⵉⴷⵔⴰⵙⵉⵏ
Ecole Nationale des Sciences Appliquées de Fès

جامعة سيدي محمد بن عبد الله
ⵜⴰⵙⴷⴰⵡⵜ ⵙⵉⴷⵉ ⵎⵓⵃⴰⵎⵎⴷ ⴱⵏ ⵄⴱⴷⵍⵍⴰⵀ
Université Sidi Mohamed Ben Abdellah

TP2 :

AWS Academy Cloud Foundations





BELHAJ KHAOULA

N 9

GSCSI- G1

Table des matières

Table des matières 2
Table de figures 4
AWS Elastic Beanstalk 5
Introduction 5
Tâche 1 : Accéder à l'environnement Elastic Beanstalk 6
1. Objectif de la tâche 6
2. Démarche suivie 6
3. Observations techniques 7
4. Conclusion de la tâche 7
Tâche 2 : Déployer un exemple d'application dans Elastic Beanstalk 8
1. Objectif de la tâche 8
2. Procédure de déploiement 8
3. Observations et Résultats 9
4. Conclusion de la tâche 10
Tâche 3 : Explorer les ressources AWS prenant en charge votre application 11
1. Objectif de la tâche 11
2. Analyse des ressources Amazon EC2 11
3. Mécanismes de Haute Disponibilité et de Réseau 12
4. Observations techniques 12
5. Conclusion de la tâche 12
Conclusion 13
Lab 4: Working with EBS 14
Introduction 14
Tâche 1 : Création d'un nouveau volume EBS 15
1. Objectif de la tâche 15
2. Analyse de l'existant 15
3. Procédure de création du volume 15
4. Résultats et Observations 16
5. Conclusion de la tâche 16
Tâche 2 : Attachement du volume à une instance 17
1. Objectif de la tâche 17

2. Procédure technique d'attachement 17
3. Observations et Changements d'état 18
4. Analyse de l'architecture 18
5. Conclusion de la tâche 19
Tâche 3 : Connexion à votre instance Amazon EC2 20
1. Objectif de la tâche 20
2. Méthodologie : Utilisation d'EC2 Instance Connect 20
3. Observations techniques 21
4. Analyse de l'importance de la tâche 21
5. Conclusion de la tâche 21
Tâche 4 : Création et Configuration du Système de Fichiers 22
1. Objectif de la tâche 22
2. Procédure technique 22
3. Observations et Vérification 23
4. Analyse technique 24
5. Conclusion de la tâche 25
Tâche 5 : Création d'un instantané Amazon EBS 26
1. Objectif de la tâche 26
2. Procédure technique 26
3. Simulation de perte de données et vérification 27
4. Analyse de la stratégie de sauvegarde 27
5. Conclusion de la tâche 27
Tâche 6 : Restauration d'un instantané Amazon EBS 28
1. Objectif de la tâche 28
2. Procédure de restauration technique 28
3. Validation des données 29
4. Analyse de la flexibilité EBS 29
5. Conclusion de la tâche 29
Conclusion 31

Table de figures

Figure 1 Environnements 6
Figure 2 Statut Ok 7
Figure 3 HTTP 404 (Not Found) 7
Figure 4 Tomcat 8
Figure 5 Validation URL 9
Figure 6 Surveillance 10
Figure 7 Security Groups 11
Figure 8 Submit 13
Figure 9 Création du volume 16
Figure 10 Available 16
Figure 11 Attach Volume 17
Figure 12 L'état du volume In-use 19
Figure 13 Connect 20
Figure 14 Audit du stockage 22
Figure 15 Création du système de fichiers 23
Figure 16 Sudo 23
Figure 17 Validation des commandes 23
Figure 18 echo 24
Figure 19 Récapitulatif df -h 24
Figure 20 sudo texte 24
Figure 21 Snapshot 26
Figure 22 sudo rm 27
Figure 23 Attachement à l'instance 28
Figure 24 Validation des données 29

AWS Elastic Beanstalk

Introduction

Dans le paysage actuel du cloud computing, la gestion de l'infrastructure est devenue une composante critique pour les entreprises souhaitant déployer des services web de manière rapide, sécurisée et évolutive. L'infrastructure traditionnelle, nécessitant une configuration manuelle complexe des serveurs, des réseaux et des politiques de sécurité, est progressivement remplacée par des solutions automatisées. Au cœur de cette transformation se trouve **AWS Elastic Beanstalk**, un service de type *Platform as a Service (PaaS)* proposé par Amazon Web Services.

L'objectif principal de cette activité pratique était de démystifier le processus de déploiement d'applications au sein de l'écosystème AWS. Contrairement aux approches basées sur le *Infrastructure as a Service (IaaS)*, où l'utilisateur doit administrer chaque couche de la pile technologique, Elastic Beanstalk automatise la gestion des ressources, depuis le provisionnement de l'infrastructure jusqu'à la mise à l'échelle (Auto Scaling) et l'équilibrage de charge (Load Balancing). Au cours de cet atelier, nous avons exploré le cycle de vie d'une application Java : du déploiement initial via un package compressé jusqu'à l'observation fine des ressources Amazon EC2 sous-jacentes. Ce rapport détaille les étapes franchies, les outils utilisés et les observations techniques permettant de comprendre comment Elastic Beanstalk orchestre, en toute transparence, la haute disponibilité et la performance d'une application web.

Tâche 1 : Accéder à l'environnement Elastic Beanstalk

1. Objectif de la tâche

L'objectif de cette première étape était de vérifier l'état de santé opérationnel de l'environnement AWS Elastic Beanstalk pré-configuré et de confirmer la connectivité initiale avec le serveur d'application avant d'y déployer un code spécifique.

2. Démarche suivie

- **Accès au service :** Accès à la console AWS via le lien sécurisé fourni, navigation vers le service *Elastic Beanstalk*.
- **Localisation de l'environnement :** Identification de l'application dans la section "Environnements" du tableau de bord.

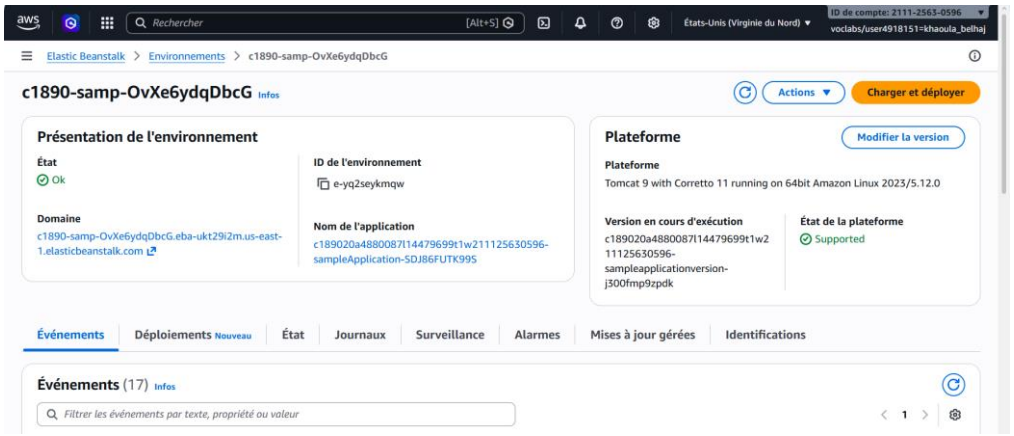


Figure 1 Environnements

- **Vérification de l'état :** Surveillance de la colonne "État" jusqu'à l'obtention du statut **Ok**, garantissant que les instances EC2 et les ressources réseau étaient pleinement opérationnelles.
- **Test de connectivité HTTP :** Accès à l'URL publique générée par le domaine `.elasticbeanstalk.com`.

3. Observations techniques

- **Statut de santé :** L'environnement a atteint le statut **Ok** en quelques minutes, confirmant que le cluster était prêt.

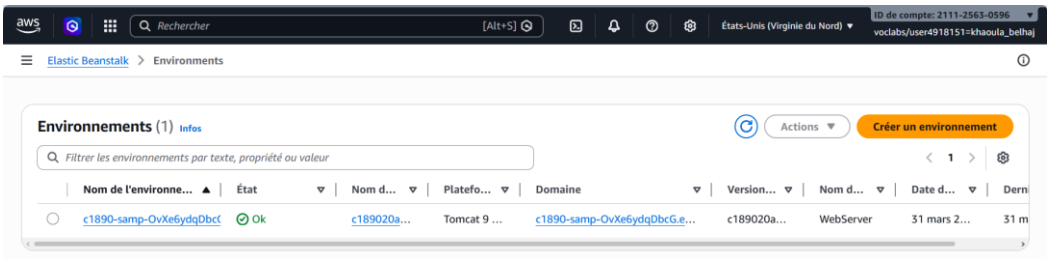


Figure 2 Statut Ok

- **Résultat du test :** Lors de l'accès à l'URL, le navigateur a renvoyé un code **HTTP 404 (Not Found)**.



Figure 3 HTTP 404 (Not Found)

  - *Analyse technique :* Cette erreur n'indique pas une défaillance du service, mais confirme que le serveur web (dans ce cas, Apache Tomcat) est en cours d'exécution et répond aux requêtes, mais qu'aucune ressource applicative (fichier .war ou .zip) n'a encore été déployée dans le répertoire racine du serveur.

4. Conclusion de la tâche

La Tâche 1 a été validée avec succès. L'infrastructure est stable et prête à accepter le déploiement de l'application `tomcat.zip`. La configuration réseau de base est fonctionnelle, puisque la réponse 404 démontre que le trafic HTTP atteint bien l'équilibreur de charge et les instances de destination.

Tâche 2 : Déployer un exemple d'application dans Elastic Beanstalk

1. Objectif de la tâche

L'objectif de cette étape consistait à injecter une application web fonctionnelle (sous forme d'archive compressée) dans l'environnement AWS Elastic Beanstalk précédemment validé. Cette phase permet de tester la capacité du service à orchestrer le déploiement de code et à mettre à jour les instances en cours d'exécution.

2. Procédure de déploiement

- **Acquisition du code source :** Téléchargement de l'archive `tomcat.zip`, un package d'exemple fourni par la documentation AWS contenant une application web compatible avec le serveur Tomcat.
- **Initialisation de la mise à jour :** Utilisation de la fonction **Charger et déployer** (*Upload and Deploy*) via le tableau de bord de l'environnement.

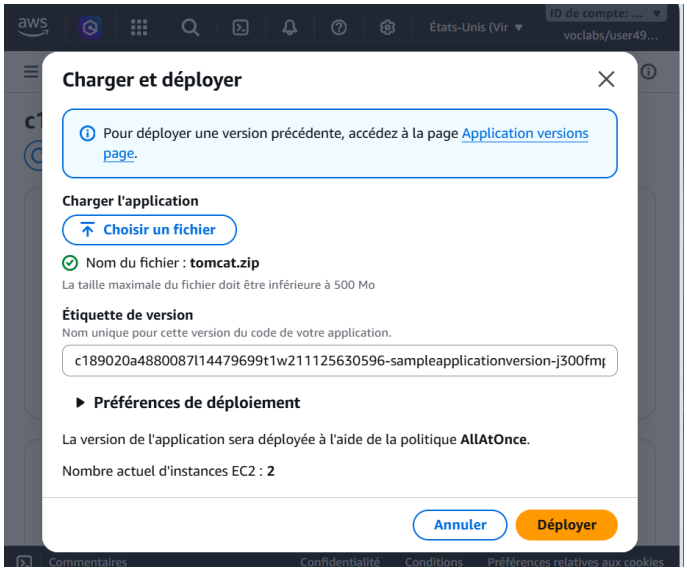


Figure 4 Tomcat

- **Configuration du package :** Sélection du fichier local et validation du déploiement vers le groupe d'instances.
- **Phase de propagation :** Observation du processus de mise à jour. Elastic Beanstalk a automatiquement :

1. Téléchargé l'archive vers un compartiment S3.
2. Déployé le code sur les instances EC2.
3. Redémarré les services nécessaires pour rendre l'application active.

3. Observations et Résultats

- **Indicateurs de santé :** Durant le déploiement, l'état de l'environnement est passé temporairement à l'état **Info** (gris) ou **Warning** (jaune) avant de redevenir **Ok** (vert) une fois l'application déployée avec succès.
- **Validation URL :** Après rafraîchissement de la page qui affichait précédemment une erreur 404, l'application web d'exemple s'est affichée correctement.

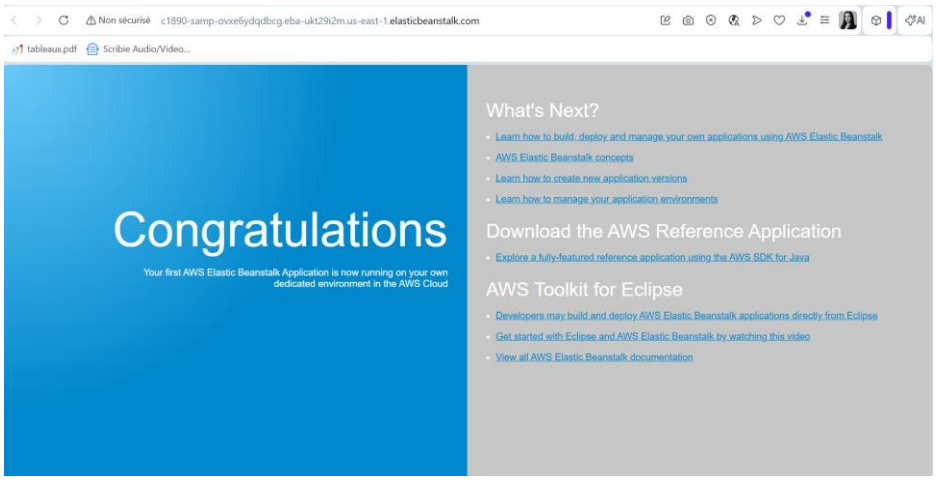


Figure 5 Validation URL

- **Analyse de la configuration :** Une inspection de l'onglet **Configuration** a permis de confirmer les paramètres de l'infrastructure :
  - **Instances :** Type et nombre d'instances Amazon EC2.
  - **Scaling :** Paramètres du groupe Auto Scaling (définissant les limites de capacité).
  - **Réseau :** Absence de base de données active, mais possibilité d'en ajouter une via les options de configuration.

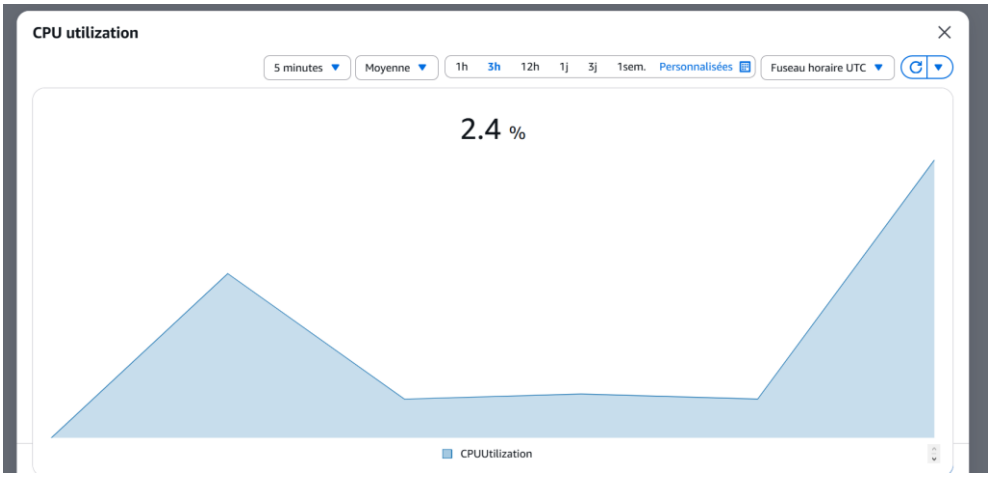


Figure 6 Surveillance

4. Conclusion de la tâche

Le déploiement a été réalisé avec succès. Cette tâche démontre l'efficacité d'Elastic Beanstalk pour gérer le cycle de vie applicatif : l'utilisateur n'a eu qu'à fournir le code, tandis que la plateforme s'est chargée de la distribution sur le cluster, de la configuration du serveur web et de la mise à jour des points de terminaison réseau.

Tâche 3 : Explorer les ressources AWS prenant en charge votre application

1. Objectif de la tâche

L'objectif de cette étape finale était de lever le voile sur l'abstraction fournie par Elastic Beanstalk afin d'identifier et de comprendre les ressources **IaaS (Infrastructure as a Service)** réellement déployées dans le compte AWS pour supporter l'application web.

2. Analyse des ressources Amazon EC2

En naviguant vers le tableau de bord **Amazon EC2**, plusieurs composants clés ont été identifiés :

- **Instances de calcul :** Présence de **deux instances EC2** en état "Running". Ces instances portent un nom généré incluant le préfixe de l'application, confirmant qu'elles sont les hôtes physiques du serveur Tomcat.
- **Sécurité (Security Groups) :** Un groupe de sécurité spécifique a été créé par Elastic Beanstalk, configuré avec des règles entrantes autorisant le trafic sur le **port 80 (HTTP)**, permettant ainsi l'accès public à l'application.

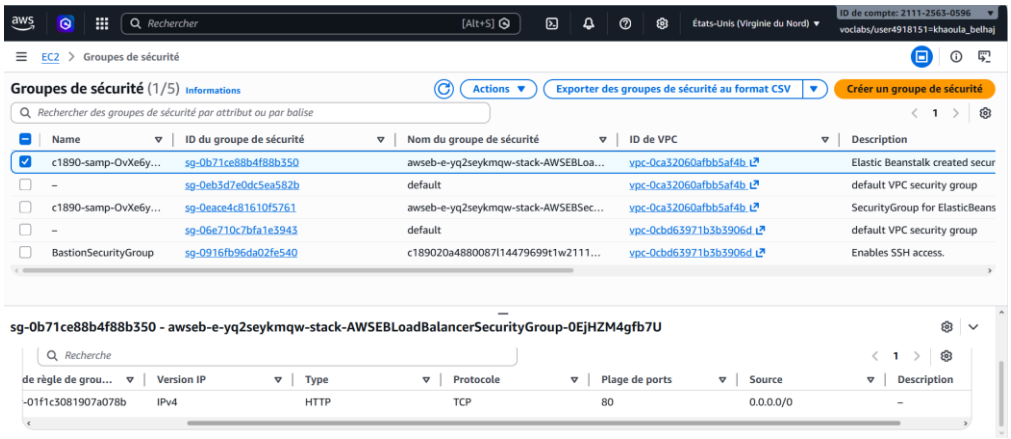


Figure 7 Security Groups

3. Mécanismes de Haute Disponibilité et de Réseau

L'exploration a permis de mettre en lumière l'architecture résiliente mise en place automatiquement :

- **Équilibreur de charge (Elastic Load Balancer) :** Un ELB est actif et configuré pour distribuer les requêtes entrantes entre les deux instances EC2. Cela garantit que si une instance échoue, le trafic est redirigé vers la seconde.
- **Groupe d'Auto Scaling (ASG) :** Cette ressource assure la gestion de la capacité. Elle est configurée pour maintenir un **minimum de 2 instances** et peut monter jusqu'à **6 instances** si la charge CPU ou réseau augmente, assurant ainsi la scalabilité de l'application.

4. Observations techniques

Ressource	Observation Clé
Instances EC2	État "Running", type d'instance optimisé pour le calcul web.
Auto Scaling	Politique de santé active (Health Check) liée à l'ELB.
Réseau	Les instances sont réparties dans différentes zones de disponibilité (AZ) pour maximiser la tolérance aux pannes.

5. Conclusion de la tâche

Cette exploration confirme qu'Elastic Beanstalk n'est pas un service "boîte noire" fermé. Bien qu'il automatise le déploiement, il s'appuie sur des services AWS standards (EC2, ASG, ELB) que l'administrateur peut continuer à surveiller et à auditer manuellement. Cette transparence permet de combiner la simplicité du déploiement PaaS avec la puissance et la granularité du contrôle IaaS.

Conclusion

En conclusion, cette activité a permis d'illustrer concrètement la valeur ajoutée de l'automatisation dans le cloud computing. L'utilisation d'AWS Elastic Beanstalk a démontré qu'il est possible de réduire drastiquement la complexité opérationnelle liée au déploiement applicatif. En déléguant la gestion du provisionnement des instances, de la configuration des groupes de sécurité et de la mise en œuvre de l'équilibrage de charge à AWS, les développeurs peuvent se concentrer sur l'essentiel : la logique métier et l'expérience utilisateur.

L'expérience a également mis en exergue la visibilité totale que conserve l'utilisateur sur les ressources déployées. Bien que le service soit géré, l'accès aux instances Amazon EC2, la possibilité de personnaliser les configurations de mise en réseau et l'analyse des données de surveillance démontrent qu'Elastic Beanstalk ne sacrifie pas le contrôle au profit de la simplicité. Ce service constitue ainsi un levier stratégique pour les organisations cherchant à concilier agilité de développement et robustesse d'infrastructure. Enfin, cette manipulation a renforcé la compréhension du fonctionnement des architectures distribuées modernes, où la résilience n'est plus une option mais une composante native, intrinsèquement liée à la conception du service lui-même.

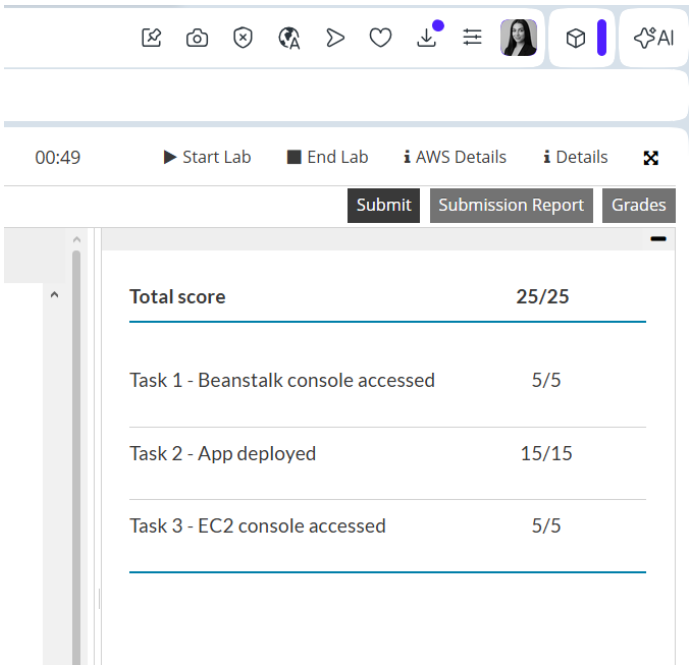


Figure 8 Submit

Lab 4: Working with EBS

Introduction

Dans l'écosystème Amazon Web Services, la persistance des données est un pilier fondamental de l'architecture des systèmes. Alors que les instances Amazon EC2 fournissent la puissance de calcul, elles nécessitent souvent un stockage externe capable de survivre à l'arrêt ou à la terminaison de l'instance. C'est ici qu'intervient **Amazon Elastic Block Store (EBS)**. Amazon EBS fournit des volumes de stockage au niveau bloc, hautement disponibles et durables, que l'on peut comparer à des disques durs virtuels connectés via le réseau.

L'objectif de cet atelier était de maîtriser le cycle de vie complet d'un volume EBS : de sa création initiale à sa sécurisation par des mécanismes de sauvegarde. Contrairement au stockage éphémère (Instance Store), EBS permet de dissocier totalement les données de la durée de vie de l'instance de calcul. Au cours de cette activité, nous avons abordé des concepts critiques tels que le montage de systèmes de fichiers sous Linux, la modification du fichier de configuration système `/etc/fstab` pour garantir la persistance au redémarrage, et l'utilisation des **Snapshots** (instantanés). Ces derniers, stockés de manière incrémentale sur Amazon S3, constituent la pierre angulaire des stratégies de reprise après sinistre (*Disaster Recovery*) et de clonage d'environnements. Ce rapport détaille les manipulations techniques effectuées pour transformer un volume brut en un espace de stockage exploitable et restaurable.

Tâche 1 : Création d'un nouveau volume EBS

1. Objectif de la tâche

L'objectif principal était de provisionner une unité de stockage par blocs (EBS) indépendante, destinée à être utilisée comme volume de données supplémentaire pour une instance Amazon EC2 existante. Cette étape permet de séparer le stockage du système d'exploitation (volume racine) des données applicatives pour une meilleure gestion de la persistance.

2. Analyse de l'existant

Avant la création du nouveau volume, une inspection de l'instance nommée **Lab** a été effectuée :

- **Identification de la Zone de Disponibilité (AZ) :** L'instance résidait dans une zone spécifique (par exemple, `us-east-1a`).
- **Observation du volume racine :** Un premier volume de **8 GiB** était déjà présent, servant de partition de démarrage pour le système d'exploitation Linux.

3. Procédure de création du volume

Le nouveau volume a été configuré avec les paramètres précis suivants pour répondre aux besoins de l'exercice :

- **Type de volume :** *General Purpose SSD (gp2)*. Ce type offre un excellent équilibre entre prix et performance pour une large gamme de charges de travail transactionnelles.
- **Taille : 1 GiB**. Une taille réduite a été choisie pour illustrer la flexibilité du service EBS qui permet de créer des volumes allant de 1 Go à 16 To.
- **Zone de Disponibilité :** La **même zone** que l'instance Lab a été impérativement sélectionnée.

  Note technique cruciale : Un volume EBS est une ressource liée à une zone spécifique. Pour qu'un volume puisse être attaché à une instance, les deux ressources doivent impérativement partager la même Zone de Disponibilité.

- **Étiquetage (Tagging) :** Ajout d'une balise `Key: Name` / `Value: My Volume` pour faciliter l'identification et la gestion des ressources dans l'inventaire AWS.

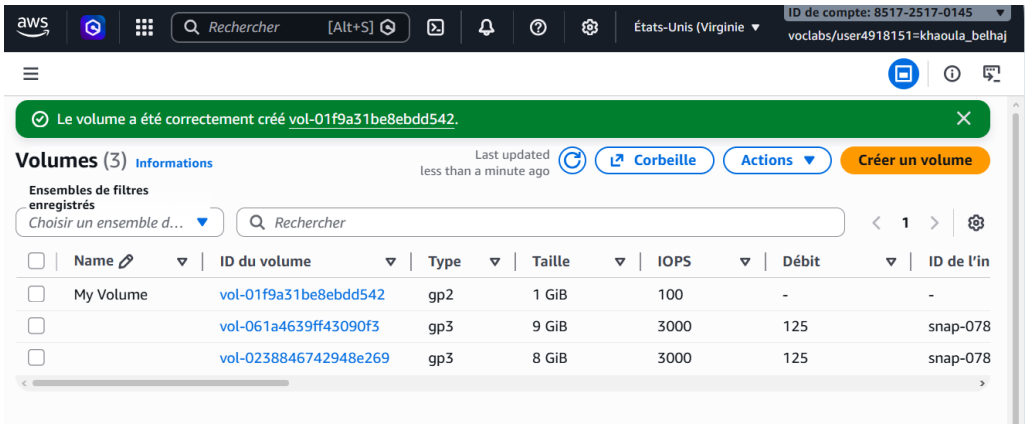


Figure 9 Création du volume

4. Résultats et Observations

- **Cycle de vie :** Après validation, le volume est passé de l'état `Creating` à l'état `Available`.

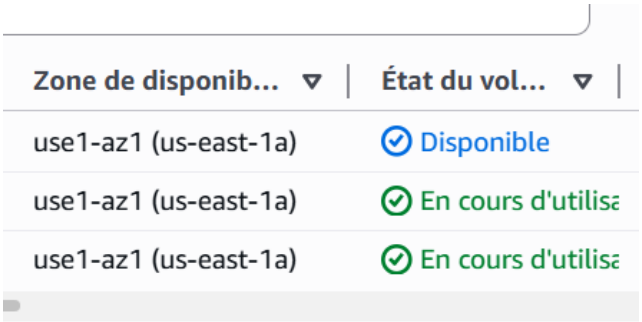


Figure 10 Available

- **Indépendance :** À ce stade, le volume existe physiquement sur l'infrastructure AWS mais n'est relié à aucune puissance de calcul. Il est "prêt à l'emploi" mais ne consomme aucune ressource CPU de l'instance.

5. Conclusion de la tâche

La Tâche 1 a été finalisée avec succès. Le volume de 1 GiB est désormais disponible dans le pool de stockage. La réussite de cette étape repose sur la cohérence géographique (AZ) entre le stockage et l'instance, une règle fondamentale de l'architecture AWS pour minimiser la latence réseau entre le calcul et les données.

Tâche 2 : Attachement du volume à une instance

1. Objectif de la tâche

L'objectif de cette opération était de connecter le volume EBS de **1 GiB** ("My Volume") à l'instance Amazon EC2 nommée **Lab**. Cette manipulation permet de transformer un volume de stockage isolé en un périphérique bloc utilisable par le système d'exploitation de l'instance, à l'image du branchement d'un disque dur externe sur un ordinateur physique.

2. Procédure technique d'attachement

La procédure a été réalisée via la console de gestion Amazon EC2 en suivant ces étapes clés :

- **Sélection de la ressource :** Identification du volume "My Volume" dont l'état était `Available` (disponible).
- **Action d'attachement :** Utilisation du menu "Actions" pour l'option **Attach Volume**.

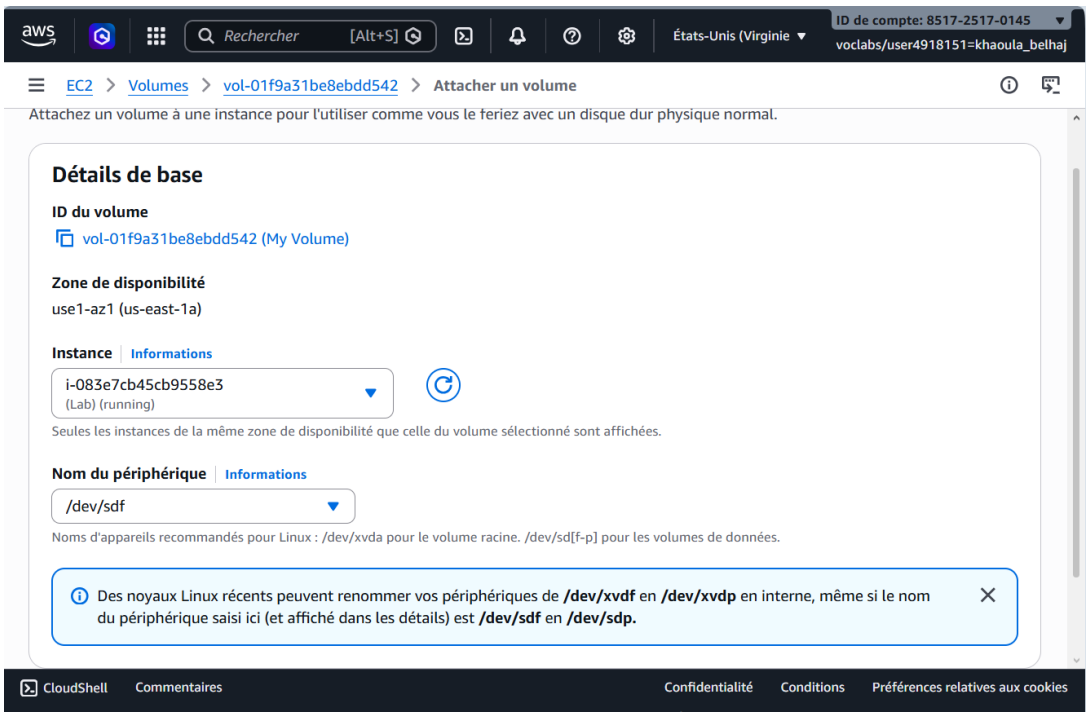


Figure 11 Attach Volume

- **Ciblage de l'instance :** Sélection de l'instance **Lab** dans la liste déroulante.

    - *Note :* Seules les instances situées dans la même Zone de Disponibilité que le volume sont apparues dans cette liste, confirmant la règle de proximité géographique établie en Tâche 1.
- **Définition du nom de périphérique (Device Name) :** Le système a proposé par défaut `/dev/sdf`.

3. Observations et Changements d'état

- **Transition d'état :** Une fois l'action validée, l'état du volume est passé de `Available` à `In-use` (en cours d'utilisation).
- **Visibilité système :** Bien que le volume soit "attaché", il est important de noter qu'à ce stade, il n'est pas encore visible comme un dossier dans l'explorateur de fichiers de l'instance. Il est reconnu par le noyau Linux comme un disque brut (raw block device).
- **Renommage interne :** Une notification de la console a précisé que les noyaux Linux récents peuvent renommer les périphériques de manière interne (par exemple, de `/dev/sdf` vers `/dev/xvdf`). Cette information est cruciale pour les étapes suivantes de configuration logicielle.

4. Analyse de l'architecture

L'attachement crée une liaison réseau à faible latence entre l'instance et le volume. Contrairement au stockage local, ce volume EBS peut être détaché de cette instance et réattaché à une autre ultérieurement, garantissant une mobilité totale des données.

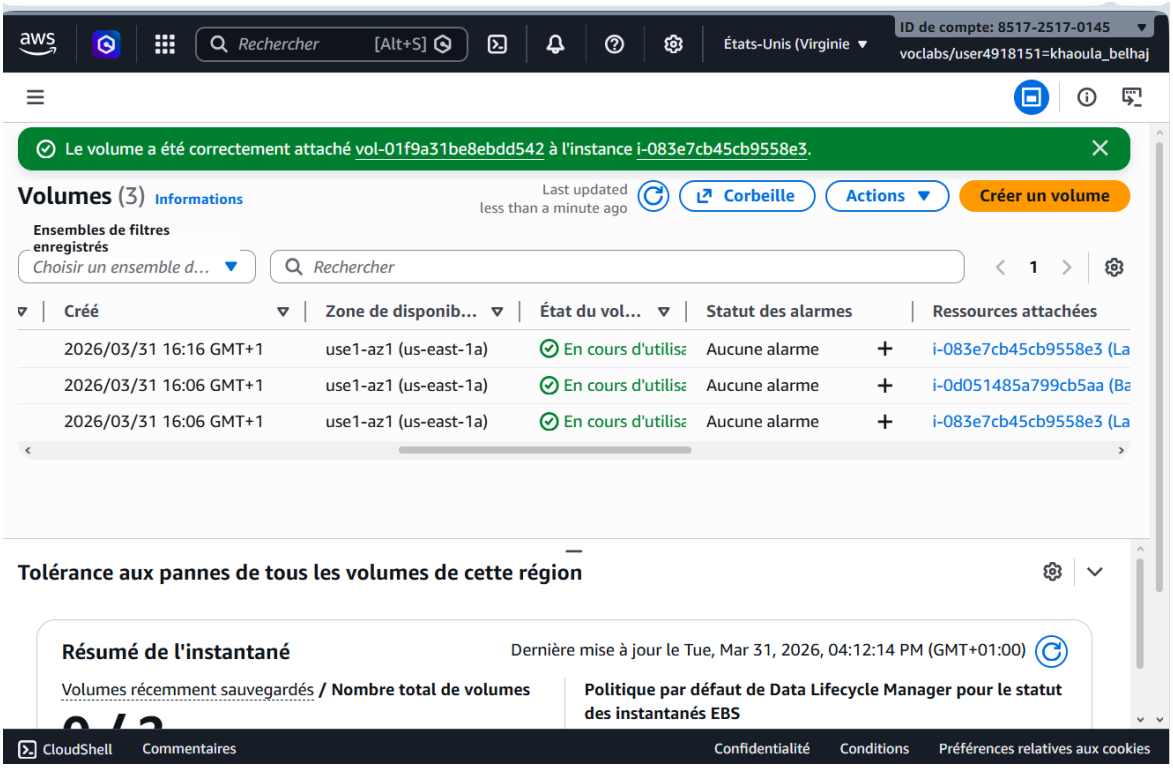


Figure 12 L'état du volume In-use

5. Conclusion de la tâche

La Tâche 2 s'est achevée avec succès. Le lien matériel virtuel est désormais établi. L'infrastructure est prête pour la phase de configuration logicielle (Tâche 3 et 4), qui consistera à formater ce nouveau disque pour que le système de fichiers Linux puisse y écrire des données de manière structurée.

Tâche 3 : Connexion à votre instance Amazon EC2

1. Objectif de la tâche

L'objectif de cette étape était d'établir une connexion à distance sécurisée avec l'instance **Lab** afin d'interagir directement avec le système d'exploitation Linux. Contrairement à une connexion SSH classique nécessitant des clés privées (`.pem`) et un client local (comme PuTTY ou un terminal), nous avons utilisé **EC2 Instance Connect**.

2. Méthodologie : Utilisation d'EC2 Instance Connect

Cette méthode présente l'avantage majeur de permettre une connexion via le navigateur web, sans nécessiter de configuration réseau complexe (comme l'ouverture du port 22 sur un pare-feu local ou la gestion de clés SSH sur la machine de l'utilisateur).

- **Accès :** Navigation dans la console EC2, section *Instances*.
- **Connexion :** Sélection de l'instance **Lab** suivie du choix de l'option **Connecter**.
- **Validation :** Utilisation de l'onglet *EC2 Instance Connect* intégré à la console. Une fois le bouton "Connect" sélectionné, un terminal interactif s'est ouvert dans l'onglet du navigateur.

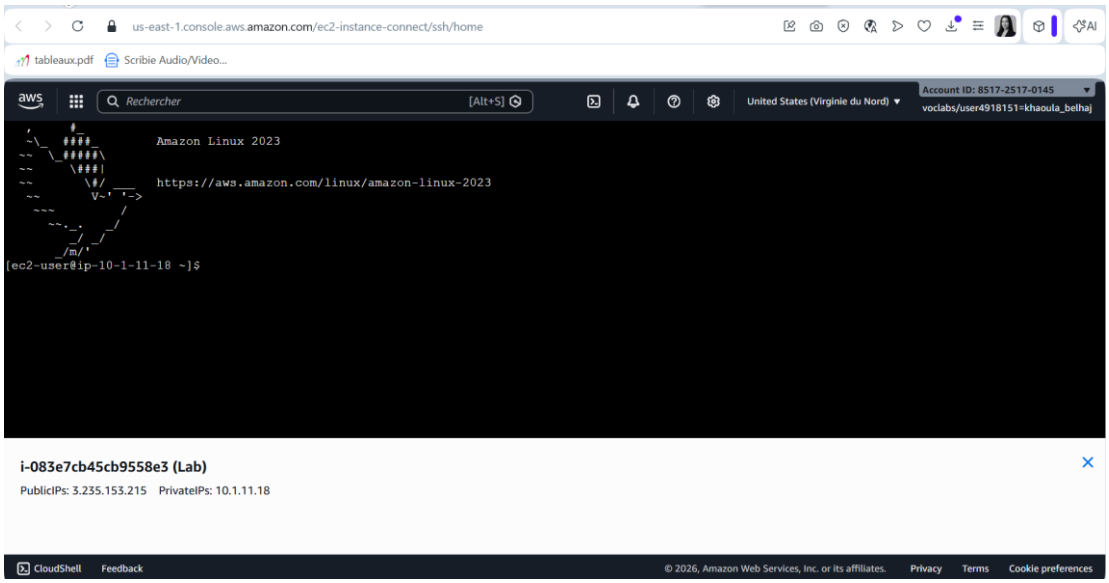


Figure 13 Connect

3. Observations techniques

- **Interface :** L'ouverture d'une session de terminal avec un prompt `$` confirme que la communication entre le service AWS (via l'agent Instance Connect) et l'instance est opérationnelle.
- **Sécurité :** Cette méthode utilise des politiques IAM (Identity and Access Management) pour valider l'accès, éliminant le besoin de stocker manuellement des clés privées sur le poste de travail.
- **État du système :** La réussite de cette connexion valide que l'instance est bien en état "Running" et que le réseau AWS est correctement configuré pour permettre le trafic entrant vers l'instance.

4. Analyse de l'importance de la tâche

Cette connexion est le pivot central de la suite des opérations. Elle permet de passer d'une gestion "externe" (via la console AWS pour attacher des volumes) à une administration "interne" (système Linux). C'est au travers de ce terminal que nous allons désormais pouvoir formater le disque et monter le système de fichiers pour rendre le stockage EBS exploitable.

5. Conclusion de la tâche

La Tâche 3 a été réalisée avec succès. La session de terminal est active, offrant un accès root (ou utilisateur privilégié) nécessaire pour effectuer les manipulations système complexes prévues dans la tâche suivante.

Tâche 4 : Création et Configuration du Système de Fichiers

1. Objectif de la tâche

L'objectif était de préparer le volume EBS de 1 GiB récemment attaché afin qu'il puisse accueillir des données. Cette étape consiste à transformer le volume brut (perçu par le noyau Linux comme un périphérique bloc sans structure) en un système de fichiers organisé (ext3) et à le monter sur un répertoire spécifique (`/mnt/data-store`) pour le rendre accessible aux applications.

2. Procédure technique

La configuration a été réalisée via la session EC2 Instance Connect en suivant ces étapes fondamentales :

- **Audit du stockage (`df -h`) :** La commande a révélé que seul le volume racine (`/dev/xvda1`) était monté. Le nouveau volume EBS n'était pas encore reconnu par le système de fichiers, confirmant qu'une action administrative était nécessaire.

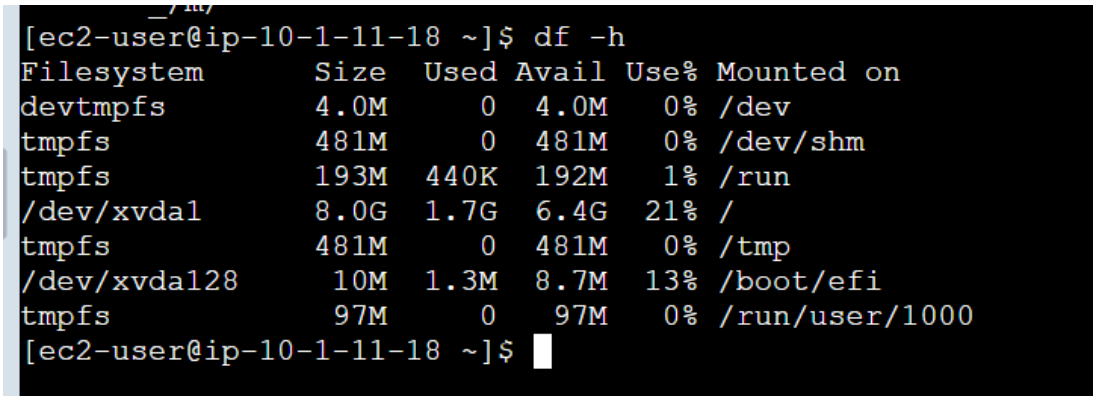


Figure 14 Audit du stockage

- **Création du système de fichiers :** Exécution de la commande `sudo mkfs -t ext3 /dev/sdf`. Cette opération a écrit la structure de métadonnées du système de fichiers ext3 sur le volume brut.

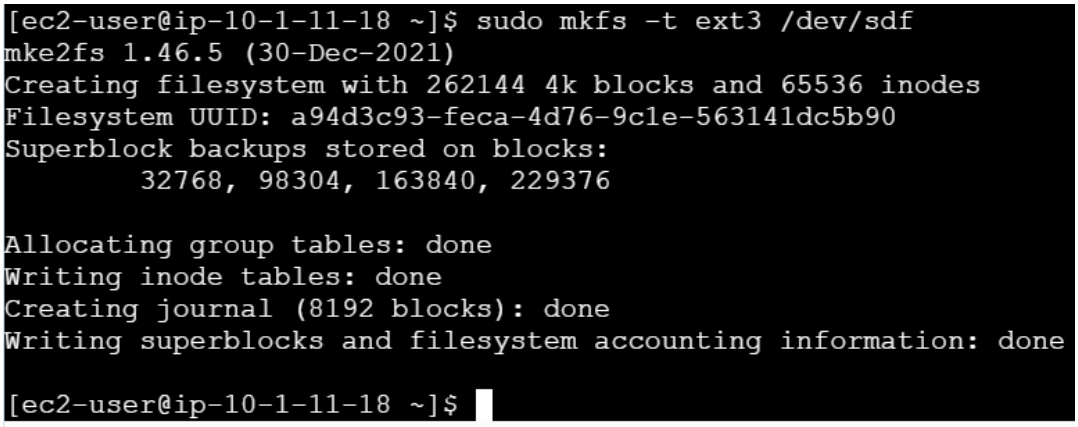


Figure 15 Création du système de fichiers

- **Préparation du point de montage :** Création du dossier cible via `sudo mkdir /mnt/data-store`.
- **Montage et Persistance :** Le volume a été monté avec `sudo mount`, puis une entrée a été ajoutée au fichier `/etc/fstab` afin d'automatiser ce montage à chaque redémarrage de l'instance.

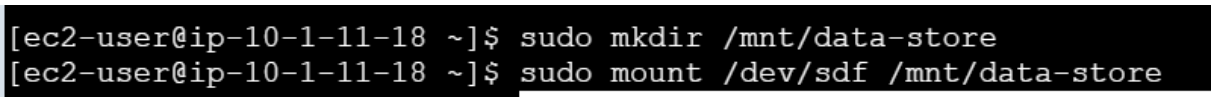


Figure 16 Sudo

3. Observations et Vérification

- **Validation des commandes :** L'utilisation de `cat /etc/fstab` a permis de confirmer l'ajout de la directive de montage, garantissant la durabilité de la configuration.

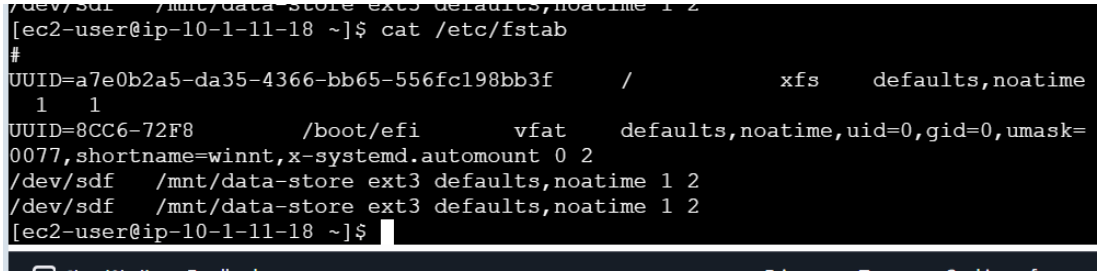


Figure 17 Validation des commandes

- **Test d'écriture :** La création du fichier `file.txt` avec la commande `sudo sh -c "echo some text..."` a prouvé que le volume est non seulement monté, mais qu'il possède les permissions d'écriture nécessaires pour une utilisation réelle.

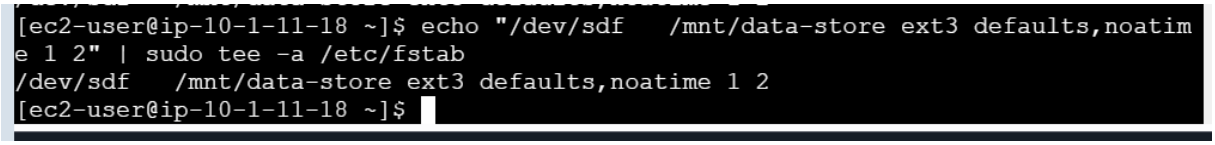


Figure 18 echo

- **Récapitulatif `df -h` :** La commande finale a affiché le nouveau volume `/dev/xvdf` monté sur `/mnt/data-store`, confirmant l'intégration réussie de l'espace de stockage de 1 GiB dans l'arborescence du système.

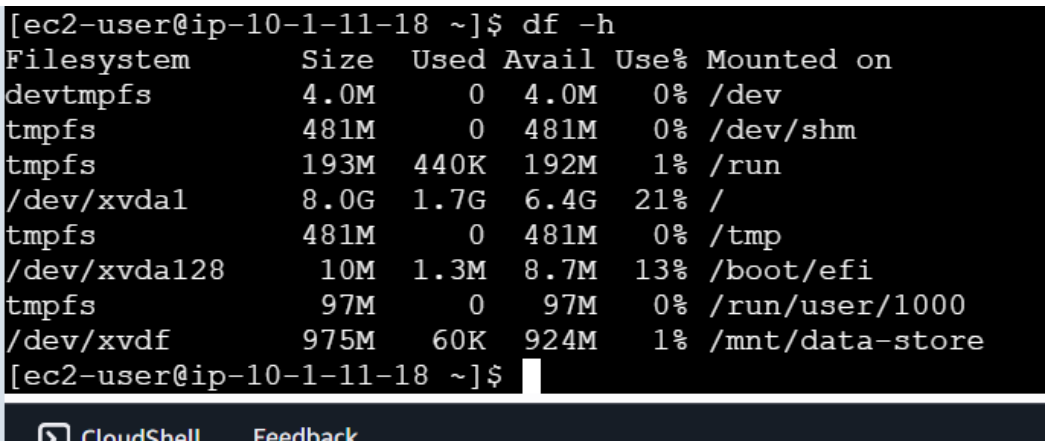


Figure 19 Récapitulatif df -h

4. Analyse technique

Le processus a illustré une règle d'or de l'administration système sous Linux : un périphérique bloc doit toujours être associé à un système de fichiers (formatage) et à un répertoire (montage) avant de pouvoir être exploité. L'utilisation du fichier `/etc/fstab` est une pratique exemplaire pour assurer la résilience de l'infrastructure face aux redémarrages forcés ou planifiés.

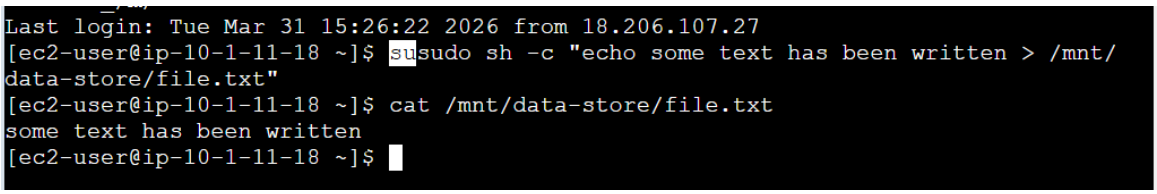


Figure 20 sudo texte

5. Conclusion de la tâche

La Tâche 4 a été réalisée avec succès. Le système de fichiers est pleinement opérationnel. Cette étape est déterminante pour la suite, car elle crée le support de données sur lequel nous allons effectuer les tests de sauvegarde et de restauration (Snapshots).

Tâche 5 : Création d'un instantané Amazon EBS

1. Objectif de la tâche

L'objectif était de réaliser une sauvegarde à un instant T (*point-in-time*) du volume EBS actif. Contrairement à une sauvegarde classique, un snapshot EBS est une copie incrémentale stockée sur Amazon S3, offrant une durabilité élevée. Cette tâche permet de garantir que les données peuvent être restaurées ultérieurement, même en cas de suppression accidentelle ou de défaillance majeure.

2. Procédure technique

La création du snapshot a été effectuée via la console Amazon EC2 :

- **Déclenchement :** Sélection du volume "My Volume" dans le tableau de bord, puis exécution de la commande **Create Snapshot**.
- **Identification :** Ajout d'une balise `Key: Name` / `Value: My Snapshot` pour assurer une traçabilité claire des sauvegardes.
- **Cycle de vie du Snapshot :**
    1. **Pending :** Le processus a commencé par une phase de copie des blocs de données utilisés vers le stockage S3.
    2. **Completed :** Une fois le transfert terminé, le snapshot est passé à l'état "Completed".

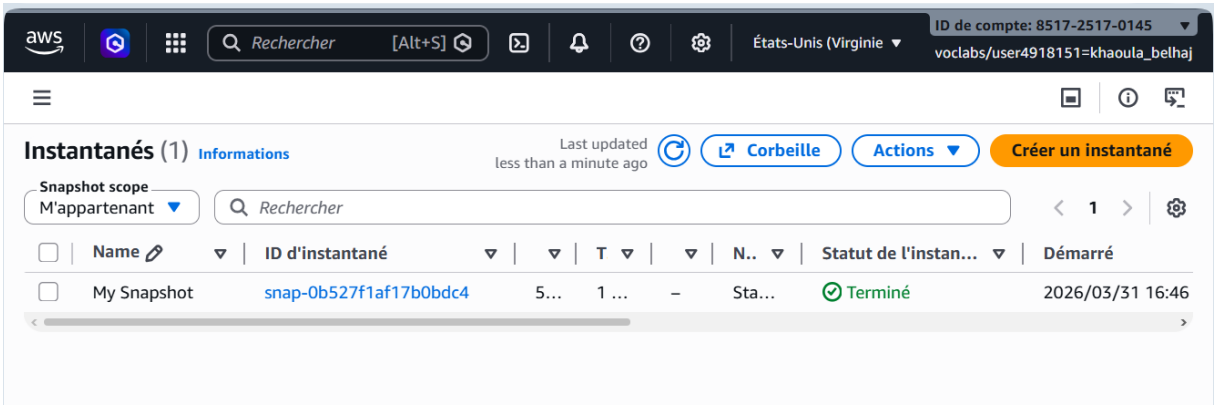


Figure 21 Snapshot

3. Simulation de perte de données et vérification

Pour valider l'intégrité de la sauvegarde :

- **Action destructive :** Dans le terminal EC2 Instance Connect, le fichier `file.txt` (créé lors de la Tâche 4) a été supprimé via la commande `sudo rm`.
- **Vérification :** La commande `ls /mnt/data-store/` a confirmé l'absence du fichier sur le volume actif.
    - *Analyse :* Le volume est désormais "vide" de ce fichier, tandis que le snapshot conservé sur S3 contient encore une copie exacte de ce fichier tel qu'il était avant la suppression.

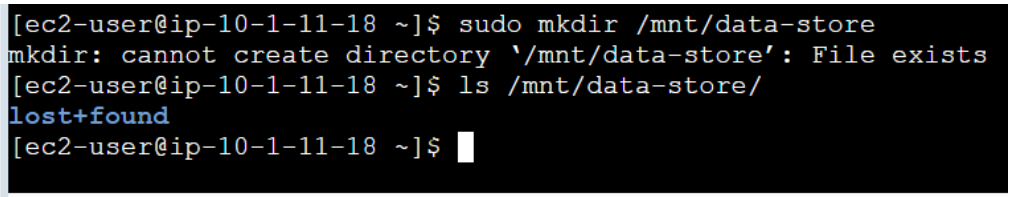


Figure 22 sudo rm

4. Analyse de la stratégie de sauvegarde

Cette tâche a mis en évidence deux caractéristiques fondamentales des snapshots EBS :

- **Efficacité de l'espace :** Seuls les blocs de données modifiés (ou occupés) sont copiés. Si vous créez plusieurs snapshots, ils ne stockent que les différences, ce qui optimise l'utilisation de l'espace sur S3.
- **Indépendance vis-à-vis du volume :** Une fois le snapshot créé, il survit même si le volume d'origine est supprimé. Il devient une entité autonome pouvant être clonée ou partagée entre régions AWS.

5. Conclusion de la tâche

La Tâche 5 a été validée avec succès. La création du snapshot démontre la capacité d'AWS à protéger les données de manière non intrusive. Cette sauvegarde servira de base de restauration pour la tâche suivante, confirmant que le plan de reprise après sinistre est fonctionnel.

Tâche 6 : Restauration d'un instantané Amazon EBS

1. Objectif de la tâche

L'objectif ultime de cette manipulation était de prouver l'efficacité de la stratégie de sauvegarde mise en place. Il s'agissait de restaurer les données supprimées accidentellement lors de la tâche précédente en créant un nouveau volume EBS à partir du snapshot "My Snapshot", puis en le rendant accessible à l'instance EC2.

2. Procédure de restauration technique

La phase de restauration s'est déroulée en trois étapes majeures dans la console et le terminal :

- **Création du volume à partir du Snapshot :**
  - Sélection de "My Snapshot" dans la console AWS.
  - Action **Create volume from snapshot**.
  - Configuration du nouveau volume ("Restored Volume") en veillant à conserver la même **Zone de Disponibilité** que l'instance.
- **Attachement à l'instance :**
  - Le nouveau volume a été attaché à l'instance **Lab**.
  - AWS lui a attribué un nouvel identifiant de périphérique, `/dev/sdg`, pour éviter tout conflit avec le disque existant (`/dev/sdf`).

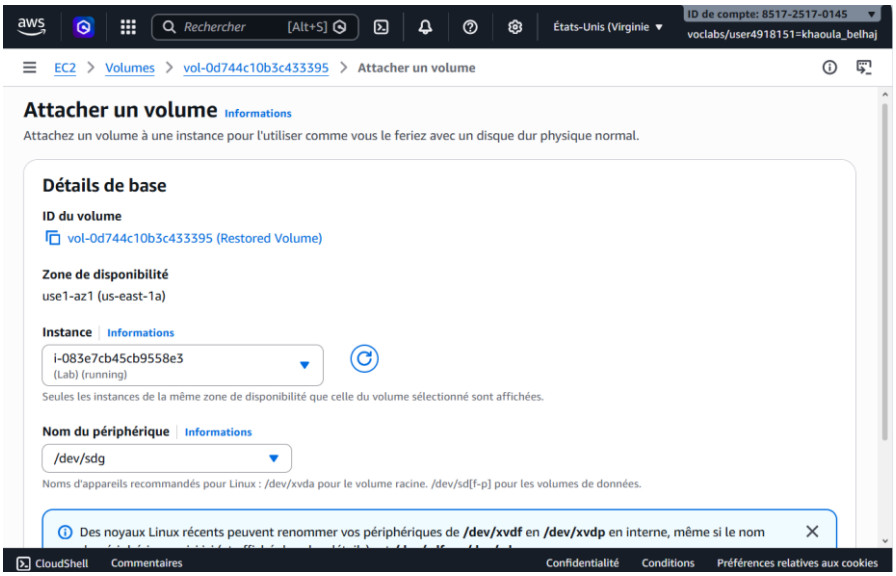


Figure 23 Attachement à l'instance

- **Montage système (Linux) :**
    - Création d'un second point de montage : `sudo mkdir /mnt/data-store2`.
    - Montage du nouveau périphérique : `sudo mount /dev/sdg /mnt/data-store2`.

3. Validation des données

Le succès de l'opération a été confirmé par l'inspection du contenu du nouveau point de montage :

- **Commande :** `ls /mnt/data-store2/`

```
[ec2-user@ip-10-1-11-18 ~]$ sudo mkdir /mnt/data-store2
[ec2-user@ip-10-1-11-18 ~]$ sudo mount /dev/sdg /mnt/data-store2
[ec2-user@ip-10-1-11-18 ~]$ ls /mnt/data-store2/
file.txt  lost+found
[ec2-user@ip-10-1-11-18 ~]$
```

Figure 24 Validation des données

- **Résultat :** Le fichier `file.txt` est apparu dans la liste, confirmant que son contenu a été préservé et restauré avec succès.
    - *Analyse :* Le volume d'origine (`/mnt/data-store`) est resté vide de ce fichier, tandis que le volume restauré (`/mnt/data-store2`) a permis de récupérer l'information perdue.

4. Analyse de la flexibilité EBS

Cette tâche a mis en lumière une fonctionnalité avancée d'Amazon EBS : la capacité de **clonage et de modification**. Lors de la restauration d'un snapshot, il est possible de créer un volume plus grand que l'original ou de changer son type (passer de gp2 à io1, par exemple), ce qui offre une grande souplesse pour faire évoluer l'infrastructure sans interruption de service.

5. Conclusion de la tâche

La Tâche 6 a été réalisée avec un succès total. Elle valide l'ensemble de la chaîne de valeur du stockage EBS : création, utilisation, protection et récupération. La capacité à restaurer des

données critiques en quelques minutes souligne la robustesse des architectures Cloud AWS face aux erreurs humaines ou aux pannes logicielles.

Conclusion

En conclusion, cet atelier a permis de démontrer la flexibilité et la robustesse du stockage **Amazon EBS** comme solution de stockage par blocs pour le Cloud. Nous avons pu constater que la gestion du stockage ne se limite pas à la simple création de ressources dans la console AWS, mais nécessite une interaction précise avec le système d'exploitation hôte (partitionnement, formatage et montage). La séparation entre le volume de démarrage (Root) et le volume de données (EBS additionnel) est une pratique exemplaire qui permet de mieux gérer les mises à jour système sans risquer de corrompre les données applicatives.

De plus, l'utilisation des snapshots illustre parfaitement la puissance de la sauvegarde moderne : rapide, incrémentale et délocalisée sur S3 pour une durabilité maximale. Cette capacité à transformer un instantané en un nouveau volume dans n'importe quelle Zone de Disponibilité souligne l'agilité qu'offre AWS pour les plans de continuité d'activité. Maîtriser EBS est donc indispensable pour tout architecte ou administrateur Cloud, car cela garantit non seulement la performance des entrées/sorties (I/O) nécessaires aux bases de données et applications, mais assure également la résilience et la portabilité des données face aux imprévus techniques.